

Cover Letter for TOSEM Submission

TyFlow: A Type-Aware Approach to Neural Code Models

Manuscript TOSEM-2026-0076 • Major Revision, decision of June 16, 2026

We thank the reviewers and the editor for their valuable comments. We have revised our paper accordingly. Every reviewer comment is quoted below in italics, followed by our response and by a summary of the changes we made in the paper; section and page numbers refer to the revised manuscript (46 pages).

Summary of changes. The revision adds 2 Java benchmarks (HumanEval-Java and TransCoder-GFG) that enlarge the Java evaluation to 186 tasks, a per-language significance analysis with sign-flip tests and bootstrap CIs (App. D), and a task-level failure taxonomy (App. C). It also adds runtime and pruning measurements of the constrained decoder (RQ2). The comparison with other generation methods now covers 3 decoder-only models of 7B–30B parameters (Table 3) and 4 reliability-oriented approaches (RQ3, Table 5). A new Limitations subsection (Sec. 6.3) discusses the encoder-decoder dependence, dynamically typed languages, and type-system expressiveness.

Response to the Editor

Editor’s comment.

All the reviewers agree that the paper investigates an important topic. However, reviewers also point out major concerns that need to be addressed and responded to, hence my recommendation for a Major Revision. In the following, I have provided some of the critical concerns raised by the reviewers.

- *Missing failure analysis*
- *Limited evaluation benchmarks*
- *Missing discussion on scalability*
- *Missing comparison with larger and more recent LLMs*
- *Missing comparison with state-of-the-art techniques*
- *Missing analysis of the system’s cost*

Details on the above points are in the reviews of the reviewers involved in the process. While preparing a Major Revision, please make sure to respond to all the other reviewers’ comments and questions, not just the points summarized above.

Response.

Thanks. Each of the 6 concerns is addressed with new experiments or new analyses, as summarized below:

- **Failure analysis.** Every test task of the 3 Java benchmarks is classified into 4 categories in App. C (pp. 43–44, Table 8), with 2 representative cases; the largest remaining failure category of TyFlow is well-typed but semantically wrong code. Detailed response: [Comment 1.6](#).
- **Benchmarks.** HumanEval-Java (146/16) and TransCoder-GFG (414/103) are added, so the Java evaluation spans 186 test tasks from 3 independent sources; the improvements recur on both, and the per-language significance analysis finds all 8 comparisons significant (App. D). Detailed responses: [Comment 1.3](#) and [Comment 2.1](#).
- **Scalability.** RQ2 measures the pruning strength of each component, the beam-exhaustion behavior, and the per-step and per-task runtime (up to $2.74\times$ the unconstrained base); the active beam never became empty in our runs, and syntactic pruning lowers the total task time. Scaling to richer type systems is discussed in the Limitations subsection. Detailed responses: [Comment 1.2](#) and [Comment 2.2](#).
- **Larger and more recent LLMs.** Zero- and few-shot pass@1 results for MiMo-7B, Qwen3-14B, and Qwen3-30B-A3B on all 4 benchmarks are added (Table 3, p. 26): these models perform well on Java, while TyFlow-2B outperforms them on SuFu under both prompting conditions. Detailed responses: [Comment 1.5](#) and [Comment 2.3](#).
- **State-of-the-art techniques.** RQ3 is rewritten as a comparison of 5 methods on MBJP (rejection sampling, SynCode, Repilot, iterative compiler repair); none of the 4 external methods significantly improves functional correctness, while TyFlow-2B is the best of all compared methods. Detailed responses: [Comment 3.1](#) and [Comment 3.2](#).
- **System cost.** The runtime analysis is reported in RQ2 together with the component ablation it belongs to: per-step and per-task wall time of the same 220M model in the 4 configurations, with an explanation of where each cost comes from and why syntactic pruning even *lowers* the per-task total. For the compared external methods, the RQ3 setup states each method’s computational budget. Detailed responses: [Comment 2.2](#) and [Comment 3.3](#).

Beyond these 6 cross-cutting concerns, we respond to every other reviewer comment and question individually in Comments 1.1–3.6 below.

Response to Reviewer 1

We thank the reviewer for the positive assessment of the formalization, its generality over grammar-based approaches, and the ablation study.

Comment 1.1.

The example language ($\lambda \rightarrow$) is extremely simple. While it serves the pedagogical purpose well, readers may reasonably wonder whether the approach scales to languages with subtyping, polymorphism, type inference, overloading, or mutable state. The paper partially addresses this by evaluating on Java, but the overview could better acknowledge the gap between $\lambda \rightarrow$ and real languages.

The restriction to first-order unification deserves further discussion. Higher-order unification arises naturally in languages with higher-order functions and polymorphism. The authors should discuss what happens when this restriction is violated and what classes of type systems are excluded.

[Weaknesses:] *The restriction to first-order unification and encoder-decoder architectures limits near-term applicability.*

Response.

Thanks. We answer the 3 questions separately.

(1) The running example and richer type systems.

TyFlow is not tied to the example language. A richer statically typed language is supported by the same method: define its program, type, and context terms, express its typing rules as constrained Horn clauses, and implement the required computable constraints. The transformation from typing rules to synthesis rules is unchanged. Because this language-specific implementation requires substantial engineering effort, we currently instantiate TyFlow only for Java and SuFu.

The difference from real languages. The example language is deliberately simple and is used only to demonstrate how TyFlow converts typing rules into synthesis rules. It contains Boolean and function types with explicitly annotated, monomorphic bindings. Unlike mainstream languages, it does not include features like subtyping, polymorphism, type inference, overloading, or mutable state, but these features can be supported in the same way as in the running example. We now acknowledge this gap explicitly in the overview.

(2) The first-order unification restriction.

Higher-order functions and polymorphism do not by themselves require higher-order unification, so TyFlow supports them. What the restriction excludes is elaboration that must infer an unknown type constructor or predicate, as in dependent types.

Why we use first-order unification. To apply a typing rule, TyFlow needs unification to terminate, decide whether the rule conclusion matches the current goal, and return a principal substitution for the remaining premises. First-order unification provides these guarantees: it is decidable

and returns a most-general unifier that is unique up to variable renaming. We therefore use first-order unification as a sufficient condition.

When higher-order unification is needed. A higher-order function takes or returns a function value, and the type checker needs to determine the type of that value. Higher-order unification is needed only when the unknown metavariable to be solved in the typing rule is itself a higher-order function, type constructor, or predicate. The following 2 terms show the difference:

$$\text{Arrow}(\alpha, \beta) \quad \text{versus} \quad F(\text{Int}) = \text{List}(\text{Int}).$$

In the first term, *Arrow* is fixed. Only α and β are unknown, so this is still a first-order problem. In the second term, *F* is such an unknown, so it is a higher-order unification problem.

ML is an example. It supports higher-order functions and parametric polymorphism, but Hindley–Milner type inference uses first-order unification. Java provides the same distinction. A simplified typing rule for a higher-order API such as `Stream.map` is:

$$\frac{\Gamma \vdash s : \text{Stream}\langle A \rangle \quad \Gamma \vdash f : \text{Function}\langle A, B \rangle}{\Gamma \vdash s.\text{map}(f) : \text{Stream}\langle B \rangle}.$$

The program passes a function as a value, but `Stream` and `Function` are fixed type constructors. Rule application only needs to instantiate the ordinary type terms *A* and *B*. Thus, this higher-order language feature still requires only first-order unification.

Support for real-world type systems. Java supports function values, but its type checking does not require unification over unknown type-level functions, type constructors, or predicates. Its program, type, and context syntax can be represented as first-order terms. The Java Language Specification handles generic inference through constraint reduction, incorporation, and resolution over inference variables.¹ Features such as subtyping, generic bounds, overload applicability, and lambda target typing can therefore be handled through typing rules and language-specific constraints. Supporting these features requires substantial implementation work, but it does not require higher-order unification as long as rule application only instantiates ordinary term variables.

Dependent-type systems such as Lean are different. Their elaboration may need to infer an unknown predicate or other higher-order entity, so applying a typing rule there requires higher-order unification. Lean handles these problems automatically with heuristics-guided elaboration: it solves the decidable fragment it recognizes and searches among the remaining candidate solutions with heuristics and backtracking.² A simple example is the induction rule for natural numbers, where the schematic parameter *P* ranges over predicates rather than terms:

$$\frac{\Gamma \vdash p_0 : P \text{ zero} \quad \Gamma \vdash p_s : \forall n. P \ n \rightarrow P \ (\text{succ } n) \quad \Gamma \vdash t : \text{Nat}}{\Gamma \vdash \text{ind}(P, p_0, p_s, t) : P \ t}$$

In Lean, *P* is implicit and absent from the surface syntax, so applying this rule means inferring the predicate. When *t* is a compound term such as `2 * n`, the candidate predicates are infinitely

¹*The Java Language Specification*, Java SE 25, Sec. 18, “Type Inference.”

²Leonardo de Moura, Jeremy Avigad, Soonho Kong, and Cody Roux. “Elaboration in Dependent Type Theory.” CADE-25, 2015.

many and incomparable, the heuristics may fail, and Lean reports “motive is not clear” until the user supplies the predicate by hand. The analogous step in TyFlow is the automatic derivation of metavariables once a rule is selected, and TyFlow currently requires this step to be deterministic: first-order unification always terminates. Following these real languages, one could instead force higher-order unification into a solvable fragment through heuristic search and branch-and-bound-style strategies. Building such a solver amounts to implementing a new unifier together with its own search strategy, which requires substantial engineering effort.

This does not conflict with the completeness of CHCs: rules with predicate parameters remain expressible as CHCs, and what first-order unification cannot provide is their automatic, deterministic application. Languages such as Java rarely create such problems, since their unknowns are ordinary types rather than predicates.

(3) The architecture restriction. The representation and synthesis rules of TyFlow can be reused in a decoder-only model. The model predicts the next rule or variable-assignment token from the prompt and previous decisions, and the synthesis engine still applies that decision, updates the derivation tree, and checks the constraints. The part that needs adaptation is the dynamic typing context: in the current encoder-decoder design, the encoder processes the changing synthesis goal separately from the decision sequence. One adaptation is to append the current typing context (the changing synthesis goal) to the decision tokens generated so far, so that the decoder sees both when predicting the next decision. Because the context changes after every decision, this appended part has to be updated at every step, and the model cannot reuse its cached computations for it. Sec. 6.3 (p. 30) describes this adaptation path; implementing it and evaluating its runtime remain future work.

Changes made.

- **Added** Sec. 2 (p. 9): the overview now enumerates the gap between $\lambda \rightarrow$ and real languages (subtyping, polymorphism, type inference, overloading resolution, mutable state) and points to the evaluation and the Limitations discussion.
- **Clarified** Sec. 3 (p. 12): why TyFlow uses first-order unification, that most languages’ typing rules require only first-order unification, and that the few languages needing more can be supported through decidable fragments with a different unifier.
- **Added** Sec. 6.3, “Dependence on the encoder-decoder architecture” (p. 30): why the dynamic typing context is not directly portable to a decoder-only model.
- **Added** Sec. 6.3, “Type-system expressiveness” (p. 30): which features can use first-order terms and which require a different solver.

Revised manuscript, Sec. 2 (p. 9) – the acknowledged gap between the example language and real languages

The $\lambda \rightarrow$ language above serves for exposition: it is deliberately minimal, containing only the base type `bool`, function types, and explicitly annotated, monomorphic bindings. Real languages additionally involve subtyping, polymorphism, type inference, overloading resolution, and mutable

state, none of which $\lambda^{\rightarrow}$ exercises. The evaluation (Sec. 6) therefore runs TyFlow on Java and SuFu instances with fuller type systems; the scope of the underlying rule framework is stated in Sec. 3, and how the approach would extend to richer type systems is discussed in Sec. 6.3.

Revised manuscript, Sec. 3 (p. 12) – the role and boundary of first-order unification

First-order unification always terminates. It either rejects two terms or returns a most-general unifier that is unique up to variable renaming. These guarantees let TyFlow apply a selected typing rule and update its remaining premises.

Supporting higher-order functions does not by itself require higher-order unification. The latter is needed only when an unknown to be solved is itself a higher-order function, type constructor, or predicate. Most languages never create such problems: their unknowns are ordinary types, and first-order unification suffices for applying their typing rules. The few languages that do, such as dependent-type systems, can be supported through a decidable fragment with principal unifiers, such as higher-order patterns, by using a different unifier, since general higher-order unification is undecidable.

Revised manuscript, Sec. 6.3, “Dependence on the encoder-decoder architecture” (p. 30) – the decoder-only adaptation path

The decision-sequence representation and synthesis rules can be used unchanged with a decoder-only model. [...] This requires recomputing the context’s key-value entries at each step and adapting cache management to the changing context. Implementing this decoding procedure and evaluating its runtime are directions for future work.

Revised manuscript, Sec. 6.3, “Type-system expressiveness” (p. 30) – type-system expressiveness as a limitation

For features that can be represented with first-order terms, such as many forms of subtyping, overloading, effects, and type classes, we need to implement the typing rules and solvers and then measure their cost. Rules requiring higher-order unification would need a suitable solver for a supported decidable fragment (Sec. 3).

Comment 1.2.

The practical implications of completeness deserve more discussion. Completeness says that for any well-typed program, there exists a synthesis derivation tree. But during generation, the LM must find the right sequence of decisions. If the LM makes a wrong choice early, it may be unable to recover. The paper does not discuss the branching factor of the search space or the likelihood of the LM making correct decisions in practice. Beam search partially addresses this, but the theoretical analysis gives no bounds on how many beams are needed.

The paper does not quantify the pruning rate in Section 5.3. What fraction of candidate tokens are eliminated at each level? This would help readers understand the practical search space reduction.

Response.

Thanks for this comment. The two paragraphs raise questions about what completeness guarantees and the pruning rates, and we answer them in turn.

(1) What completeness guarantees, and the observed beam-search behavior.

Completeness bounds what our method can reach, not how likely the search is to reach it, and

we claim no bound on the latter. The vanilla text-based tokenization and code generationh can not guarantee the target program can be found either. Relative to it, we have not narrowed the search space of programs, and we have improved how well the search works. We have also added the questions and limitations raised here to the completeness discussion (Sec. 4.4, p. 20).

What completeness guarantees. The theorem (Sec. 4.4, p. 20) states that for every well-typed program there exists a synthesis derivation tree from which that program is extracted. The guarantee is about reachability: the encoding and the synthesis rules exclude no well-typed program. The program the theorem promises to exist therefore remains reachable at every step. What the theorem does not say is where the model ranks that derivation, or how wide a beam has to be to keep it.

How the search is improved. Within that unchanged search space, TyFlow raises the chance that the search finds a correct program in two ways. The type analysis is explicit: a vanilla code generation model emits program text token by token and must maintain the type reasoning process implicitly, whereas TyFlow emits a decision sequence in which each step selects a synthesis rule and the variables that rule requires. The type reasoning is settled by the rule’s premises rather than left to the model. The explicit representation also gives us lightweight pruning: when a typing rule fails to apply or a constraint check fails, the corresponding branches are cut. So the engine detects a wrong decision as soon as the model makes it and removes that path from the beam, reducing the chance that the search ends without a correct program.

The beam-search behavior we observe. We measure the branching factor as the number of active hypotheses present at each decoding step after pruning. With beam 10 it averages 7.97, below 10 because finished or rejected candidates do not fill active slots. And in the evaluated run the active beam never became empty.

(2) Pruning rates. Instrumenting the decoder on the SuFu tasks (beam 10, 330-step budget) gives: syntactic pruning removes 59.1% of all vocabulary proposals, and type pruning a further 8.2% (type checking is applied only to the high-probability candidates about to enter the beam, which is why its rate is lower). A final, complete type check on finished derivations rejects 21.2% of the completed candidates.

Changes made.

- **Added** Sec. 4.4 (p. 20): the two theorems as a statement about the search space, neither weakened nor strengthened, together with how the representation, model training, and pruning improve the search within that space.
- **Added** Sec. 6.2.2 (p. 27): “Whether the constraints over-prune”: what completeness does and does not guarantee, the theoretical possibility of beam exhaustion, and the observed beam-10 behavior.
- **Added** Sec. 6.2.2 (p. 27): “How much the constraints prune”: instrumented rates (59.1% syntactic, 8.2% type, 21.2% final check) and the 330-step budget.

Revised manuscript, Sec. 4.4 (p. 20) – the two theorems as a statement about the search space

The two theorems characterize the search space of TyFlow: they guarantee that it is neither weakened nor strengthened, but say nothing about how the search proceeds inside that space. That is what the decision-sequence representation, model training, and the pruning techniques are for: they make the search more effective without moving its boundary.

Revised manuscript, Sec. 6.2.2, “Whether the constraints over-prune” (p. 27) – completeness and the observed beam-10 behavior

Completeness guarantees that a well-typed program has a derivation, but it does not guarantee that the model ranks a correct prefix within a fixed beam or provide a bound on the required beam size. A finite beam can discard all correct prefixes, and the retained hypotheses may later have no valid continuation, leaving the active beam empty. [...] With beam 10, this number averages 7.97. [...] In our experiments, however, the active beam never became empty.

Revised manuscript, Sec. 6.2.2, “How much the constraints prune” (p. 27) – the measured pruning rates

To measure the pruning strength, we instrument the constrained decoder on the SuFu tasks (beam 10, 330-step budget): syntactic pruning removes 59.1% of all vocabulary proposals, and type pruning a further 8.2%. The lower rate is expected: type checking is more expensive, so it is applied only to the high-probability candidates about to enter the beam. Such candidates are rarely ill-typed, yet pruning them matters, for they would otherwise expand within the beam and crowd out well-typed derivations. Type checking during search is provisional (only completed AST fields are checked); the whole program is checked once when the derivation finishes, and this final check rejects 21.2% of the completed candidates.

Comment 1.3.

The SuFu test set has only ~58 tasks (20% of 290), and the Java test set has only ~61 tasks (10% of 608). These are very small test sets, and the results may not be statistically robust. The authors should report confidence intervals or conduct significance tests. A difference of 1–2 tasks passing or failing could shift pass@1 by several percentage points.

[Weaknesses:] *Small test sets raise concerns about statistical reliability. [...] Missing failure analysis and confidence intervals.*

Response.

Thanks for this comment. We address the dataset size, the statistical uncertainty, and the missing failure analysis separately.

(1) Expanded evaluation.

Benchmark coverage. We add 2 Java benchmarks from sources independent of MBJP. HumanEval-Java contains short algorithmic methods adapted from HumanEval, with a fixed split of 146 training and 16 test tasks. TransCoder-GFG contains numeric, string, and array problems drawn from the GeeksForGeeks programs released with TransCoder, with a fixed split of 414 training and 103 test tasks. Together with the original 67 MBJP test tasks, these additions increase the Java evaluation from 67 to 186 test tasks across 3 benchmarks. SuFu remains a 58-task test set because the source repository contains 290 programs.

Experimental protocol. We evaluate the 2 added benchmarks at the 2B scale, comparing the baseline T5Gemma2-2B with our TyFlow-2B. Both systems use the same Java training tasks and are evaluated on the same fixed test tasks with beam 10 and 10 candidates per task. We report pass@1, pass@10, FSP, and CER, so the evaluation covers functional correctness, candidate ranking, and compilation errors.

Results. The 2 added benchmarks show improvements on all 4 metrics:

- **HumanEval-Java.** pass@1 improves from 31.25% to 50.00%, pass@10 from 37.50% to 56.25%, and FSP from 6.50 to 4.75; CER falls from 24.38% to 1.30%.
- **TransCoder-GFG.** pass@1 improves from 19.42% to 30.10%, pass@10 from 33.01% to 46.60%, and FSP from 7.10 to 5.81; CER falls from 13.69% to 2.75%.

The large CER reductions are consistent with the purpose of type-constrained generation, while the pass-rate and FSP gains show that the benefit is not limited to rejecting uncompileable outputs.

(2) Confidence intervals and significance tests. New App. D (p. 45) evaluates statistical significance per language: the per-task paired differences of the 220M and 2B comparisons are pooled and tested with a two-sided sign-flip randomization test, with 95% CIs from a task-level cluster bootstrap. All 8 combined differences favor TyFlow, with CIs excluding zero. On SuFu, pass@1 improves by 12.1 pp on average (95% CI [2.6, 21.6], $p = 0.025$), pass@10 by 16.4 pp ([6.0, 26.7], $p = 0.004$), FSP drops by 1.63 positions ($p = 0.001$), and CER drops by 0.79 ($p < 10^{-4}$). On Java, pass@1 improves by 9.1 pp on average ([3.2, 15.0], $p = 0.004$), pass@10 by 11.5 pp ([5.5, 17.4], $p = 4 \times 10^{-4}$), FSP drops by 1.00 positions ($p = 3 \times 10^{-4}$), and CER drops by 0.23 ($p < 10^{-4}$). Every metric difference also favors TyFlow in each individual language–scale block (Table 2).

(3) Failure analysis. Most tasks that TyFlow-2B does not solve at pass@1 are compilable but behaviorally incorrect; on MBJP, 38 of the 50 unsolved tasks fall in this category, and it is also the largest remaining category on the other 2 benchmarks. App. C (pp. 43–44) classifies every test task of the 3 Java benchmarks into 4 mutually exclusive categories and compares TyFlow-2B with its baseline in each; the second pattern is ranking failure, where a correct candidate exists among the 10 candidates but is not ranked first.

Changes made.

- **Added** Sec. 6.1.1 (p. 24) and App. B (p. 40): the sources, task characteristics, and fixed splits of HumanEval-Java and TransCoder-GFG.
- **Added** Sec. 6.2.1 (p. 25): the results of TyFlow-2B and its T5Gemma2-2B baseline on HumanEval-Java and TransCoder-GFG.
- **Added** App. D (p. 45): the per-language significance analysis with sign-flip tests and bootstrap CIs (Table 9).

- **Added** App. C (pp. 43–44): the 4-category failure analysis of all 3 Java benchmarks, with the TyFlow-2B/baseline comparison and representative cases.

Revised manuscript, Sec. 6.1.1 (p. 24) – the two added Java benchmarks and their fixed splits

HumanEval-Java and TransCoder-GFG are two additional Java benchmarks that broaden the evaluation beyond MBJP. HumanEval-Java adapts the HumanEval problems to Java [...], and TransCoder-GFG collects the Java programs released with TransCoder [...]; both follow the task format of MBJP. HumanEval-Java uses a fixed split of 146 training and 16 test tasks, and TransCoder-GFG uses a fixed split of 414 training and 103 test tasks.

Revised manuscript, App. B (p. 40) – the sources and task characteristics of the two added benchmarks

HumanEval-Java is the Java port of HumanEval [...]: short Python-style algorithm snippets (string/list manipulation, digit arithmetic, parity and divisibility tests, a few short geometry/number-theory problems) transcribed into a single Java method per task. TransCoder-GFG reuses the GeeksforGeeks Java programs released with TransCoder [...]: one- or two-sentence specifications (numeric, string, or array algorithms; standard-library types only) paired with a self-contained Java method. Tab. 7 summarizes the three Java evaluation splits used in this paper.

Revised manuscript, Sec. 6.2.1 (p. 25) – the results on the expanded benchmark set

TyFlow improves code generation accuracy at both scales and on every benchmark: [...] pass@1 rises [...] 31.25% to 50.00% (HumanEval-Java), and 19.42% to 30.10% (TransCoder-GFG) at 2B; pass@10 improves by 7.46–20.69 points on the same six benchmarks. [...] TyFlow also drops CER to 0.00% on SuFu and 0.45% / 1.30% / 2.75% on the three Java benchmarks; the small residual Java CER comes from static-analysis errors the type system does not capture.

Revised manuscript, App. D (p. 45) – the per-language significance analysis

We evaluate statistical significance per language: for each of Java and SuFu, the per-task paired differences of the 220M and 2B comparisons are pooled and tested jointly with a two-sided sign-flip randomization test (200,000 sign flips). Under the null hypothesis, the sign of each paired difference is exchangeable, and the flips automatically account for the correlation induced by tasks shared between the two scales. Pooling the scales per language is needed because each individual language-scale block (58–186 tasks) is too small for a well-powered separate test. [...] all eight combined differences favor TyFlow, with 95% CIs excluding zero.

Revised manuscript, App. C (p. 43) – the failure taxonomy and its 4 categories

This section classifies the failing tasks of the Java evaluation and presents representative cases. For each benchmark and model, every test task is assigned to one of four mutually exclusive categories: Solved@1 (the top-1 candidate passes all tests); Ranking failure (some candidate in the top ten passes, but not the top-1 one); All-invalid (no candidate passes and none of the ten candidates compiles); and Well-typed but wrong (no candidate passes, but at least one candidate compiles—the failure is semantic, not syntactic).

Comment 1.4.

On Java, TyFlow-220M improves pass@1 from 10.45% to only 11.94%, less than 1.5 percentage points absolute. While pass@10 improves more substantially, the pass@1 gain is marginal, given the test set size. The paper should discuss why Java gains are smaller.

[Weaknesses:] *The Java results are modest, and the subset restriction introduces potential selection bias.*

Response.

Thanks for this comment. The Java results show gains of TyFlow in both functional correctness and compilability. We explain below why the improvement is smaller at pass@1 and how the other results help assess its value.

(1) Why the Java gains differ from SuFu. TyFlow improves code generation by preventing type errors during generation. Java is a mainstream programming language for which the pretrained baseline already has a strong foundation, whereas SuFu is a low-resource language. Consistent with this difference, the 220M Java baseline produces fewer compilation errors than the SuFu baseline (CER 35.52% versus 83.10%). This leaves fewer errors for our type constraints to address and therefore less room for improvement on Java.

(2) The value of the Java improvements. For a mainstream language such as Java, the improvement is substantial given that TyFlow and the baseline use the same training data. At 220M, TyFlow improves pass@10 by 7.46 percentage points (35.7% relative), a larger absolute gain than the 2.80-point improvement reported by GrammarT5-base over CodeT5-base on Java CONCODE, also at 220M.³ Gains of this size are typical for retraining-based approaches on Java: on CONCODE, AST-T5 improves the exact match of CodeT5-base by 0.6 points, and StructCoder by 0.1.⁴ Our expanded experiments confirm this on 2 additional Java benchmarks: at 2B, pass@1 improves from 31.25% to 50.00% on HumanEval-Java and from 19.42% to 30.10% on TransCoder-GFG, with all 4 metrics improving on both benchmarks (pass@10 from 37.50% to 56.25% and from 33.01% to 46.60%), and the per-language significance analysis finds all 4 Java differences statistically significant (App. D).

(3) Selection Bias. We do not find evidence that our results are driven by selection bias. Our Java tasks were selected by grammar rather than by model performance, both methods are run on the same tasks, and App. B defines the supported subset the evaluation draws from. The grammar subset does not favor TyFlow. The language features we keep are the ones the baseline already handles well (Sec. 6.2.1). Those we omit, such as multithreading, lambda expressions,

³Qihao Zhu et al. “GrammarT5: Grammar-Integrated Pretrained Encoder-Decoder Neural Model for Code.” ICSE 2024, Table 4. These gains are measured on different tasks and metrics.

⁴Linyuan Gong, Mostafa Elhoushi, and Alvin Cheung. “AST-T5: Structure-Aware Pretraining for Code Generation and Understanding.” ICML 2024: in its unified CONCODE comparison, the exact match is 22.3 for CodeT5-base, 22.9 for AST-T5, and 22.4 for StructCoder. Sindhu Tipirneni, Ming Zhu, and Chandan K. Reddy. “StructCoder: Structure-Aware Transformer for Code Generation.” ACM TKDD 2024. These gains are measured on different tasks and metrics.

and reflection, are rarer in the training data, so the baseline is more likely to make mistakes on them, where our type constraints have the most to fix.

Changes made.

- **Clarified** Sec. 6.2.1 (p. 25): why the 220M Java baseline’s lower compilation-error rate leaves less room for improvement than on SuFu.
- **Added** Sec. 6.2.1 (p. 25): the 2B results on MBJP and the 2 additional Java benchmarks.
- **Clarified** App. B (p. 37): the Java grammar coverage and the task selection used in the evaluation.

Revised manuscript, Sec. 6.2.1 (p. 25) – why Java leaves less room for improvement than SuFu

At 220M, the Java baseline has a lower CER than the SuFu baseline (35.52% versus 83.10%), leaving fewer compilation errors for type constraints to address. [. . .]

Revised manuscript, Sec. 6.2.1 (p. 25) – the Java results at 2B

TyFlow improves code generation accuracy at both scales and on every benchmark: pass@1 rises [. . .] 13.43% to 25.37% (MBJP), 31.25% to 50.00% (HumanEval-Java), and 19.42% to 30.10% (TransCoder-GFG) at 2B; pass@10 improves by 7.46–20.69 points on the same six benchmarks.

Revised manuscript, App. B (p. 37) – the implemented Java grammar and evaluated tasks

The implemented Java subset supports classes, methods, variables, arrays, generic types, operators, and control flow statements; it omits multithreading, lambda expressions, and complex reflection mechanisms. [. . .] Evaluation uses only tasks built from constructs this grammar supports, so the reported results apply to this supported subset.

Comment 1.5.

The paper compares only with fine-tuned CodeT5 and T5Gemma2 variants. The absence of any comparison with modern LLMs is a notable gap. The authors could at least report zero-shot pass rates for state-of-the-art proprietary and open-weight LLMs on the same benchmarks to contextualize the absolute numbers.

[Weaknesses:] *No comparison with modern decoder-only large-scale code models.*

Response.

Thanks for this comment. We added the requested comparison on 3 larger open-weight decoder-only models (MiMo-7B, Qwen3-14B, Qwen3-30B-A3B); the full per-cell numbers are in Table 3 (p. 26). The *zero-shot* prompt is identical to the prompt given to T5Gemma2-2B and TyFlow-2B in the Sec. 6.2.1 benchmark runs (the task specification only). The *few-shot* prompt takes that same task prompt and prepends 3 solved examples for the Java benchmarks, 6 for SuFu. Each prompted model produces one greedy candidate per task. None of the 3 prompted models is fine-tuned on our training data; only T5Gemma2-2B and TyFlow-2B are fine-tuned.

- **SuFu (zero-shot).** All 3 prompted models score 0/58. Few-shot lifts at most one of them to 8/58, while TyFlow-2B scores 21/58. The decision-sequence representation helps the most where the prompted models have effectively no prior on the language.
- **Java.** The 3 prompted models (MiMo-7B, Qwen3-14B, Qwen3-30B-A3B) solve 34–42 of 67 MBJP tasks, 3–14 of 16 HumanEval-Java tasks, and 38–52 of 103 TransCoder-GFG tasks at pass@1 zero-shot (each range gives the lowest and highest scores of the 3 models). In comparison, TyFlow-2B solves 17/67, 8/16, and 31/103 respectively. Few-shot prompting only nudges these numbers: the same 3 models move to 33–45 on MBJP, 10–14 on HumanEval-Java, and 43–52 on TransCoder-GFG (Tab. 3). The flat few-shot curve on Java suggests these models may have seen these benchmarks during their own pre-training; we therefore flag a possible data-leakage risk for the Java comparison.

Changes made.

- **Added** Table 3 (p. 26): pass@1 of MiMo-7B, Qwen3-14B, and Qwen3-30B-A3B on all 4 benchmarks, zero-shot and few-shot, with the prompt protocol in the Sec. 6.2.1 setup text and the table note.
- **Added** Sec. 6.2.1 (p. 26): the SuFu results and the Java pass@1 comparison with TyFlow-2B, using task prompts without examples for the larger models, together with the data-leakage caveat.

Revised manuscript, Sec. 6.2.1, setup (p. 26) – the setup of the decoder-only comparison

To contextualize absolute performance, Tab. 3 reports three larger decoder-only models (MiMo-7B, Qwen3-14B, Qwen3-30B-A3B), prompted with the same task prompt in all conditions, alongside our two fine-tuned 2B encoder-decoder models. Each prompted model produces one greedy candidate per task: the zero-shot condition uses only the task prompt, and the few-shot condition prepends three solved examples for the Java benchmarks and six examples for SuFu.

Revised manuscript, Sec. 6.2.1, results (p. 26) – the decoder-only results on Java and SuFu

On SuFu the comparison favors TyFlow: none of the three solves a single task zero-shot, and few-shot prompting reaches at most 8 of 58, whereas TyFlow-2B solves 21 [. . .] On Java, given the task prompt without examples, the three larger models solve 34–42 of 67 MBJP tasks, 3–14 of 16 HumanEval-Java tasks, and 38–52 of 103 TransCoder-GFG tasks at pass@1. TyFlow-2B solves 17/67, 8/16, and 31/103, respectively. [. . .] we therefore flag a possible data-leakage risk for the Java comparison.

Comment 1.6.

What do the remaining failing programs look like? Are they semantically wrong but well-typed? Do they fail on edge cases? A qualitative analysis of failure modes would significantly strengthen the paper.

Response.

Thanks for this comment. Most tasks not solved by TyFlow-2B at pass@1 have compilable candidates whose behavior does not match the task specification; some contain a correct candidate that is not ranked first. We analyze these outcomes in App. C (pp. 43–44), comparing TyFlow-2B with its baseline on all 3 Java benchmarks using 4 mutually exclusive categories.

The 4 categories are:

1. **All-invalid:** no candidate compiles.
2. **Well-typed but wrong:** at least one candidate compiles, but none passes all tests.
3. **Ranking failure:** a correct candidate exists among the 10, but it is not ranked first.
4. **Solved@1:** the top-1 candidate passes all test cases.

Failure analysis of each category:

- **All-invalid.** On MBJP, the baseline produces no compilable candidate for 4 tasks, while TyFlow produces at least one compilable candidate for every task.
- **Well-typed but wrong.** Of the 50 MBJP tasks that TyFlow does not solve, 38 fall into this category; it is also the largest remaining category on HumanEval-Java and TransCoder-GFG. App. C.2 illustrates how a candidate can satisfy the implemented typing rules but fail to match the task specification; such errors need not be limited to edge cases.
- **Ranking failure.** This occurs on 12 MBJP tasks, 1 HumanEval-Java task, and 17 TransCoder-GFG tasks: TyFlow finds a solution within its 10 candidates, but not as the first one, so these tasks count as successes at pass@10 but not at pass@1.
- **Solved@1.** TyFlow improves functional correctness as well as compilability. On MBJP, the first candidate passes all tests for 17 tasks, compared with 9 under the baseline. The same improvement appears on HumanEval-Java, from 5 solved tasks to 8, and on TransCoder-GFG, from 20 to 31.

Changes made.

- **Added** App. C (p. 43): the 4-category taxonomy and Table 8 comparing the 2B models on all 3 Java benchmarks, with an explanation of compilable-but-incorrect candidates and ranking failures.
- **Added** App. C.1–C.2 (p. 44): representative compilation and semantic failures, with the distinction between general logic errors and edge-case failures clarified in C.2.
- **Added** Sec. 6.2.1 (p. 25): a pointer to the failure analysis when discussing candidate ranking and compilable but incorrect programs.

Revised manuscript, App. C (p. 43) – the failure-category results

On MBJP, the baseline produces no compilable candidate for four tasks, while TyFlow produces at least one for every task. On HumanEval-Java, neither model has a task with no compilable candidate; on TransCoder-GFG, both models have two such tasks. Most tasks not solved by TyFlow at pass@1 have at least one compilable candidate but no candidate passing all tests. On MBJP, for example, this accounts for 38 of the 50 tasks not solved at pass@1; it is also the largest remaining failure category on the other two benchmarks. In other cases, a correct candidate is generated but not ranked first: this occurs on 12 MBJP tasks, one HumanEval-Java task, and 17 TransCoder-GFG tasks. These tasks count as successes at pass@10 but not at pass@1.

Revised manuscript, App. C.1, representative case (p. 44) – the representative compilation failure case

Without the brace, the loop body is a single statement, the closing brace terminates the method early, and javac reports `illegal start of type` at the `return` statement. The model produces plausible token-level continuations, but nothing prevents such structural corruption.

Revised manuscript, App. C.2, representative case (p. 44) – the representative semantic failure case

The candidate satisfies the implemented typing rules, but its behavior does not match the task specification. This case illustrates a general logic error rather than a failure confined to edge cases; the taxonomy does not separately count edge-case failures.

Revised manuscript, Sec. 6.2.1 (p. 25) – the pointer to failure analysis

The failure analysis in Appendix C identifies ranking errors and compilable but incorrect programs among the remaining failures.

Response to Reviewer 2

We thank the reviewer for the positive assessment of the theoretical foundation, the framework design, and the ablation study.

Comment 2.1.

W1: Limited evaluation benchmarks. The evaluation uses only a limited number of SuFu test tasks and Java test tasks. These are very small test sets, and no confidence intervals or statistical significance tests are reported. In addition, the SuFu language is highly niche. While this is positioned as a “low-resource” evaluation, it’s unclear how findings generalize beyond this specific setting. There is no evaluation on standard, widely-used benchmarks. While the approach requires language-specific tooling (type checker, parser), the absence of comparison on common benchmarks could limit the paper’s impact and make it harder to contextualize the results.

(Addresses W1) Report confidence intervals or evaluate over multiple random train/test splits to establish statistical stability given the small test sets. Consider adding at least one more widely-used benchmark to help contextualize the results.

Response.

Thanks for this comment. We added 2 Java benchmarks and a per-language significance analysis with sign-flip tests and bootstrap CIs. We answer your concerns below with the complete settings and results.

(1) Evaluation on additional benchmarks. We add 2 Java benchmarks from sources independent of MBJP and from widely used benchmark collections. HumanEval-Java contains short algorithmic methods adapted from HumanEval, with a fixed split of 146 training and 16 test tasks. TransCoder-GFG contains numeric, string, and array problems drawn from the GeeksForGeeks programs released with TransCoder, with a fixed split of 414 training and 103 test tasks. Together with the original 67 MBJP test tasks, they enlarge the Java evaluation to 186 test tasks from 3 independent sources.

We evaluate the 2 added benchmarks at the 2B scale, comparing the baseline T5Gemma2-2B with our TyFlow-2B. Both systems use the same Java training tasks and are evaluated on the same fixed test tasks with beam 10 and 10 candidates per task. We report pass@1, pass@10, FSP, and CER, so the evaluation covers functional correctness, candidate ranking, and compilation errors. On both added benchmarks, TyFlow-2B improves over its baseline on all 4 metrics. Pass@1 rises from 31.25% to 50.00% on HumanEval-Java and from 19.42% to 30.10% on TransCoder-GFG; pass@10 rises from 37.50% to 56.25% and from 33.01% to 46.60%; FSP falls from 6.50 to 4.75 and from 7.10 to 5.81; CER falls from 24.38% to 1.30% and from 13.69% to 2.75%.

Regarding SuFu itself: it remains a 58-task test set because the source repository contains 290 programs; it is included as a low-resource language on which the pretrained baseline has

almost no prior (a compilation-error rate of 83.10% at 220M), so it isolates the benefit of the decision-sequence representation.

(2) Statistical reliability. New App. D (p. 45) evaluates statistical significance per language: the per-task paired differences of the 220M and 2B comparisons are pooled and tested with a two-sided sign-flip randomization test, with 95% CIs from a task-level cluster bootstrap. All 8 combined differences favor TyFlow, with CIs excluding zero. On SuFu, pass@1 improves by 12.1 pp on average (95% CI [2.6, 21.6], $p = 0.025$), pass@10 by 16.4 pp ([6.0, 26.7], $p = 0.004$), FSP drops by 1.63 positions ($p = 0.001$), and CER drops by 0.79 ($p < 10^{-4}$). On Java, pass@1 improves by 9.1 pp on average ([3.2, 15.0], $p = 0.004$), pass@10 by 11.5 pp ([5.5, 17.4], $p = 4 \times 10^{-4}$), FSP drops by 1.00 positions ($p = 3 \times 10^{-4}$), and CER drops by 0.23 ($p < 10^{-4}$). Every metric difference also favors TyFlow in each individual language-scale block (Table 2).

Changes made.

- **Added** Sec. 6.1.1 (p. 24) and App. B (p. 40): HumanEval-Java and TransCoder-GFG and their construction.
- **Added** Sec. 6.2.1 (p. 25) and Table 2 (p. 26): the results on the 2 added Java benchmarks, including the pass@10 improvements reported above.
- **Added** App. D (p. 45): the per-language significance analysis with sign-flip tests and bootstrap CIs (Table 9).

Revised manuscript, Sec. 6.1.1 (p. 24) – the two added benchmarks and their splits

HumanEval-Java and TransCoder-GFG are two additional Java benchmarks that broaden the evaluation beyond MBJP. HumanEval-Java adapts the HumanEval problems to Java, and TransCoder-GFG collects the Java programs released with TransCoder; both follow the task format of MBJP. HumanEval-Java uses a fixed split of 146 training and 16 test tasks, and TransCoder-GFG uses a fixed split of 414 training and 103 test tasks.

Revised manuscript, Sec. 6.2.1 (p. 25) – improvements on the added Java benchmarks

TyFlow improves code generation accuracy at both scales and on every benchmark: [...] pass@1 rises [...] 31.25% to 50.00% (HumanEval-Java), and 19.42% to 30.10% (TransCoder-GFG) at 2B; pass@10 improves by 7.46–20.69 points on the same six benchmarks.

Revised manuscript, App. D (p. 45) – the per-language significance analysis

We evaluate statistical significance per language: for each of Java and SuFu, the per-task paired differences of the 220M and 2B comparisons are pooled and tested jointly with a two-sided sign-flip randomization test (200,000 sign flips). Under the null hypothesis, the sign of each paired difference is exchangeable, and the flips automatically account for the correlation induced by tasks shared between the two scales. Pooling the scales per language is needed because each individual language-scale block (58–186 tasks) is too small for a well-powered separate test. [...] all eight combined differences favor TyFlow, with 95% CIs excluding zero.

Comment 2.2.

W2: Scalability concerns are insufficiently addressed.

The paper does not discuss how the synthesis derivation tree scales with program complexity. Real-world programs could have deep type derivation trees with polymorphism, subtyping, type inference, and overloading resolution, but the Java subset avoids most of these. It is also unclear what runtime overhead the synthesis system imposes during inference, as the dynamic re-encoding at each step and the unification/constraint-checking machinery introduce costs that are never quantified.

(Addresses W2) Report runtime inference time comparisons between TyFlow and the baselines. Discuss, even qualitatively, how the synthesis system would scale to richer type features such as polymorphism with variance, subtyping hierarchies, or ownership types. Report how frequently beam search exhausts all candidates due to pruning and what the fallback behavior is.

Response.

Thanks for this comment. We explain the size of the encoded derivation, the measured runtime cost of each component, and the behavior of beam search below.

(1) Synthesis-tree encoding and richer type systems. Our representation does not make the program significantly longer. We encode a synthesis derivation tree as a sequence of decisions in the order its goals are resolved. Each rule selection is represented by 1 token. Variable assignments are encoded as sequences of constructor and constant tokens. The model therefore generates tokens representing the decisions needed to reconstruct the tree, rather than the full text of every typing judgment. A rule token incorporates syntax that would otherwise require separate code tokens, such as punctuation. The encoded sequences are comparable in length to ordinary code tokens (410 vs. 405 tokens per program on SuFu; 129 vs. 142 on Java).

The same encoding applies when the type system is extended, so a larger rule set does not itself cause an explosion in sequence length of our synthesis trees. Features that can be represented with first-order terms, such as many forms of subtyping, overloading, effects, and type classes, require implementing their typing rules and solvers, but the transformation from typing rules to synthesis rules is unchanged, and each added rule still uses 1 token per application. Sequence length depends on how many rule applications and variable assignments the derivation requires. Rules requiring higher-order unification would need a different solver for a supported decidable fragment (Sec. 3, Sec. 6.3).

(2) Runtime overhead of each component. Our ablation study uses the 220M model on SuFu in 4 cumulative configurations: the base model without checks, then adding syntactic pruning, type pruning, and the dynamic typing context in turn. A decoding step extends the active hypotheses by 1 token, including the enabled checks and context update. In this order, average time per step is 31.1 → 32.4 → 43.7 → 59.6 ms. Syntactic pruning is inexpensive: checking whether a proposed token fits the expected syntax adds only 1.3 ms per step. Type pruning adds 11.3 ms because it invokes the type checker on candidate extensions; encoding the updated

typing context adds a further 15.9 ms because the evolving synthesis goal must be processed by the encoder.

Average time per task is $5.70 \rightarrow 5.34 \rightarrow 11.07 \rightarrow 15.62$ s in the same configuration order. The overall trend is similar: type checking and context encoding account for most of the additional cost. Syntactic pruning is the exception. By eliminating invalid decisions early, it keeps search on well-formed derivations that finish in fewer steps. This reduction outweighs its small per-step overhead, lowering average task time from 5.70 to 5.34 s.

(3) Beam-search behavior. TyFlow does not guarantee that the model always ranks a correct program within a fixed beam (that’s what we want our trained model to do). A finite beam can therefore discard all correct continuations. If the active beam is exhausted, search stops and retains any completed candidates; if none has been completed, the run produces no solution. But in our runs the active beam never became empty; the number of active hypotheses within the beam present at each decoding step averages 7.97 at beam size 10.

Changes made.

- **Added** Sec. 6.2.4 and Table 6 (p. 29): the token-length comparison between decision sequences and ordinary code; Sec. 6.3 (p. 30) discusses richer type-system extensions.
- **Added** Sec. 6.2.2 and Table 4 (pp. 27–28): the runtime measurements for the 4 component configurations, including average per-step and per-task times.
- **Added** Sec. 6.2.2 (p. 27): the discussion of finite-beam search and the observed active-beam size (7.97 on average, with no empty active beam in the instrumented run).

Revised manuscript, Sec. 6.2.4 (p. 29) – decision-sequence length compared with code tokens

It is even comparable to, or slightly shorter than, the plain code tokens alone (405 on SuFu, 142 on Java): a type derivation rule absorbs the redundant syntactic elements of the code (such as equals signs and semicolons) into a single rule token, so our representation adds little and sometimes removes tokens.

Revised manuscript, Sec. 6.2.2, “Runtime cost” (p. 28) – the runtime measurements

Each component increases the per-step cost ($31.1 \rightarrow 32.4 \rightarrow 43.7 \rightarrow 59.6$ ms). Syntactic pruning adds little (roughly +4%); type pruning and the dynamic typing context cost considerably more, as the former invokes the type checker at every step and the latter additionally re-encodes the evolving synthesis goal. The per-task totals move by -6.38% / $+94.00\%$ / $+173.88\%$ ($2.74\times$ base). [. . .]

Revised manuscript, Sec. 6.2.2 (p. 27) – the beam-exhaustion observation

A finite beam can discard all correct prefixes, and the retained hypotheses may later have no valid continuation, leaving the active beam empty. [. . .] We measure the effective branching factor as the number of active hypotheses present at each decoding step after pruning. With beam 10, this number averages 7.97. [. . .] In our experiments, however, the active beam never became empty.

Revised manuscript, Sec. 6.3, “Type-system expressiveness” (p. 30) – the scope of richer type-system extensions

For features that can be represented with first-order terms, such as many forms of subtyping, overloading, effects, and type classes, we need to implement the typing rules and solvers and then measure their cost. Rules requiring higher-order unification would need a suitable solver for a supported decidable fragment (Sec. 3). [...] Implementing and evaluating these extensions remains future work.

Comment 2.3.

W3: Comparison with modern LLMs is absent. The baselines are CodeT5-220M and T5Gemma2-2B, and both are relatively small encoder-decoder models. There is no comparison with decoder-only models which are much more common in today’s community. Additionally, no model larger than 2B parameters is evaluated. While the paper demonstrates clear improvements over its chosen baselines, it remains an open question whether larger models might already mitigate type errors sufficiently through scale alone. This would contextualize the practical value of the proposed approach.

(Addresses W3) Include at least a zero-shot or few-shot evaluation of a modern decoder-only model on the same benchmarks. Additionally, discuss whether the core ideas could be adapted to decoder-only architectures, even if full integration is left to future work.

Response.

Thanks for this comment. In our evaluation, TyFlow shows consistent improvements over its baselines even the 2B baseline makes less type mistakes than the 220m baseline. We will further report the larger-model comparison in full, then explain how TyFlow could be adapted to a decoder-only model.

(1) The comparison with larger decoder-only models. We added the comparison on 3 larger open-weight decoder-only models (MiMo-7B, Qwen3-14B, Qwen3-30B-A3B); the full per-cell numbers are in Table 3 (p. 26). The *zero-shot* prompt is identical to the prompt given to T5Gemma2-2B and TyFlow-2B in the Sec. 6.2.1 benchmark runs (the task specification only). The *few-shot* prompt takes that same task prompt and prepends 3 solved examples for the Java benchmarks, 6 for SuFu. Each prompted model produces one greedy candidate per task. None of the 3 prompted models is fine-tuned on our training data; only T5Gemma2-2B and TyFlow-2B are fine-tuned.

- **SuFu (zero-shot).** All 3 prompted models score 0/58. Few-shot lifts at most one of them to 8/58, while TyFlow-2B scores 21/58. The decision-sequence representation helps the most where the prompted models have effectively no prior on the language.
- **Java.** The 3 prompted models (MiMo-7B, Qwen3-14B, Qwen3-30B-A3B) solve 34–42 of 67 MBJP tasks, 3–14 of 16 HumanEval-Java tasks, and 38–52 of 103 TransCoder-GFG tasks at pass@1 zero-shot (each range gives the lowest and highest scores of the 3 models). In comparison, TyFlow-2B solves 17/67, 8/16, and 31/103 respectively. Few-shot prompting

only nudges these numbers: the same 3 models move to 33–45 on MBJP, 10–14 on HumanEval-Java, and 43–52 on TransCoder-GFG (Tab. 3). The flat few-shot curve on Java suggests these models may have seen these benchmarks during their own pre-training; we therefore flag a possible data-leakage risk for the Java comparison.

(2) Adapting TyFlow to a decoder-only model. The decision-sequence representation can be used unchanged: a decoder-only model can predict the next rule or variable-assignment token from the prompt and previous decisions. The synthesis engine would still apply that decision, update the derivation tree, and check the constraints. The part that needs adaptation is the dynamic typing context. In the current encoder-decoder design, the encoder processes the changing synthesis goal separately from the decision sequence.

One adaptation is to append the current typing context (the changing synthesis goal) to the decision tokens generated so far, so that the decoder sees both when predicting the next decision. Because the context changes after every decision, this appended part has to be updated at every step, and the model cannot reuse its cached computations for it. Thus, the representation and synthesis rules carry over, while context input and cache management require changes to the decoding procedure. Sec. 6.3 describes this adaptation direction; implementation and runtime evaluation remain future work.

Changes made.

- **Added** Table 3 (p. 26) and Sec. 6.2.1 (p. 26): the decoder-only comparison.
- **Added** Sec. 6.3, “Dependence on the encoder-decoder architecture” (p. 30): what a decoder-only adaptation requires.

Revised manuscript, Sec. 6.2.1, setup (p. 26) – the setup of the decoder-only comparison

To contextualize absolute performance, Tab. 3 reports three larger decoder-only models (MiMo-7B, Qwen3-14B, Qwen3-30B-A3B), prompted with the same task prompt in all conditions, alongside our two fine-tuned 2B encoder-decoder models. Each prompted model produces one greedy candidate per task: the zero-shot condition uses only the task prompt, and the few-shot condition prepends three solved examples for the Java benchmarks and six examples for SuFu.

Revised manuscript, Sec. 6.2.1, results (p. 26) – the decoder-only results on Java and SuFu

On SuFu the comparison favors TyFlow: none of the three solves a single task zero-shot [. . .] On Java, given the task prompt without examples, the three larger models solve 34–42 of 67 MBJP tasks, 3–14 of 16 HumanEval-Java tasks, and 38–52 of 103 TransCoder-GFG tasks at pass@1. TyFlow-2B solves 17/67, 8/16, and 31/103, respectively.

Revised manuscript, Sec. 6.3, “Dependence on the encoder-decoder architecture” (p. 30) – the decoder-only adaptation path

The decision-sequence representation and synthesis rules can be used unchanged with a decoder-only model. One adaptation would append the current typing context to the decision tokens generated so far as temporary input for predicting the next decision, then replace it with the updated context before the following prediction. This requires recomputing the context’s key-value entries at each step and

adapting cache management to the changing context. Implementing this decoding procedure and evaluating its runtime are directions for future work.

Comment 2.4.

W4: The CHC generality claim is overclaimed relative to the evidence. The paper repeatedly emphasizes that CHCs can encode “any Turing-computable function” and that the framework generalizes to “arbitrary logical constraints.” However, the evaluation only covers type correctness for two languages with relatively simple type systems. No evidence is provided for richer constraints. The gap between the theoretical generality and empirical validation is large.

(Addresses W4) Either provide a case study demonstrating a constraint richer than basic type correctness (e.g., ownership, effects, refinement types), or explicitly scope the generality claims to what is empirically demonstrated and acknowledge the gap as a limitation.

Response.

Thanks for this comment. The two type systems in our evaluation use different categories of constraints, which gives the evaluation sufficient diversity and representativeness. But we have also tightened overly general statements in the paper, explicitly scoping the generality claims to what is empirically demonstrated.

SuFu is a functional language: its typing rules cover inductive type declarations, recursive definitions with pattern matching, and the label/unlabel/rewrite constructs specific to its optimization framework. Java, in contrast, is an imperative, class-based language: its typing rules cover classes and methods, arrays, generic types, and implicit conversions such as auto-unboxing. The two rule sets restrict largely different constructs, which gives our evaluation sufficient diversity and representativeness.

We went through the paper and tightened statements that were more general than what we evaluate, including the two statements you quote. The introduction and related work now state the evaluated scope in the terms quoted below; the CHC-expressiveness statement in Sec. 3 now describes the formalism itself; and Sec. 8 presents richer constraint families as a direction for future work. Sec. 6.3 also states what supporting richer type features requires.

Changes made.

- **Clarified** Sec. 1 (p. 4): the CHC-level construction, its solver requirements, and its evaluation for type correctness in Java and SuFu.
- **Tightened** Sec. 3 (p. 12): the CHC-expressiveness statement is now about the formalism itself.
- **Added** Sec. 6.3, “Type-system expressiveness” (p. 30): the implementation and cost evaluation required to support richer type features, including effects.
- **Clarified** Sec. 7.2 (pp. 30–31): the extension from grammar-based representations to type-correctness proofs.

- **Clarified** Sec. 8 (p. 31): richer constraint families as a direction for future implementation and evaluation.

Revised manuscript, Sec. 1 (p. 4) – the scoped CHC claim in the introduction

Notably, all proof rules in our system are formulated as constrained Horn clauses (see Sec. 3.1.1). This representation subsumes prior work on syntactic correctness and provides a common basis for constructing synthesis rules from proof rules. The construction applies to rules in the supported CHC form with suitable unification and constraint-checking procedures. We instantiate and evaluate the framework for type correctness in Java and SuFu.

Revised manuscript, Sec. 3 (p. 12) – the tightened CHC-expressiveness statement

It is well known that any Turing-computable function can be encoded using Horn clauses (HCs) [. . .]. Consequently, CHCs are at least as expressive as HCs, and any Turing-computable function can be expressed using CHCs.

Revised manuscript, Sec. 6.3, “Type-system expressiveness” (p. 30) – requirements for richer type features

For features that can be represented with first-order terms, such as many forms of subtyping, overloading, effects, and type classes, we need to implement the typing rules and solvers and then measure their cost.

Revised manuscript, Sec. 7.2 (pp. 30–31) – the extension from grammar-based representations to type-correctness proofs

Grammar-based encoding can be viewed as a special case of our framework when grammar rules are expressed as CHCs. TyFlow extends this rule-based representation from syntactic structure to type-correctness proofs, with implementations and evaluations in Java and SuFu.

Revised manuscript, Sec. 8 (p. 31) – the scoped CHC claim in the conclusion

Beyond type correctness, TyFlow’s CHC foundation suggests a route toward richer constraint families, such as safety verification and the generation of other structural data; instantiating and evaluating these extensions remains future work.

Response to Reviewer 3

We thank the reviewer for the positive assessment of the typing-based extension of grammar-guided generation and of the rigor of the isomorphism between type derivations and synthesis derivations.

Comment 3.1.

Missing comparisons against token-level constrained decoding works such as SynCode and Copiloting the Copilots. Those works also guide LLM generation, but they are better than rejection sampling because they guide the LLM at the token level. To demonstrate the robustness of TyFlow, comparisons against those works are important.

Response.

Thanks for this comment. We added comparisons using SynCode and Repilot (*Copiloting the Copilots*) on MBJP, alongside rejection sampling and iterative compiler repair; the full results are in Table 5 (p. 28). The experiments compare these decoding-time mechanisms with TyFlow’s type-guided generation; the implementations and evaluated configurations are described below.

SynCode. We reuse SynCode’s released grammar-masking code and Java grammar with T5Gemma2 on MBJP. SynCode masks tokens that would violate the grammar at each decoding step to ensure syntactically well-formed outputs. With SynCode’s grammar masking enabled, CER falls from 26.12% to 21.34%, while the solved sets remain unchanged.

Repilot. We reuse Repilot’s modified Eclipse JDT completion engine and adapt it to T5Gemma2 on MBJP. During generation, the partial Java program is maintained as a JDT document, and completion queries guide token pruning. Following the original Repilot’s policy, punctuation and keywords bypass these queries. CER changes from 26.12% to 25.97%, and the solved sets remain unchanged.

Rejection sampling. This method repeatedly draws candidates from the model and keeps those that pass Java compilation, for up to 10 compilable candidates per task. It is the only external method that increases the number of solved tasks, adding 3 solved tasks at pass@10 and 1 at pass@1, but neither difference is statistically significant.

Pruning during decoding is partial by design: grammar masking and completion queries check less than final compilation, so both methods still leave uncompileable candidates during generation. The uncompileable candidates within the beam may occupy the slots that correct candidates would otherwise fill, causing correct candidates to be lost. Rejection sampling filters invalid Java programs only after generation, so it never loses a correct program: any candidate the model produces is kept unless it fails to compile. Early pruning therefore offers no advantage over rejection sampling here.

TyFlow-2B solves 17/67 at pass@1 and 29/67 at pass@10 with only 3/670 compilation errors, the best result among all compared methods. TyFlow enforces the modeled type constraints during generation and reduces CER to 0.45%, rather than filtering or repairing what the model already produces. When the model assigns too little probability to correct programs, decoding-time constraints alone offer limited gains; training with a suitable representation can be more effective for improving the learned distribution.

Changes made.

- **Rewrote** Sec. 6.2.3 (p. 28): the comparison of external generation mechanisms with TyFlow and the results in Table 5.
- **Added** Sec. 6.2.3 (p. 28): reuse of the released SynCode and Repilot/JDT components and how their grammar masking and completion-guided token pruning are applied to MBJP generation.

Revised manuscript, Sec. 6.2.3 (p. 28) – the five-method setup and shared protocol

On MBJP, four such methods are applied on top of the same frozen T5Gemma2-2B checkpoint selected in RQ1, adding their own pruning or repair mechanisms on top: *rejection sampling*, *SynCode*, *Repilot*(*Copiloting the Copilots*), and *iterative compiler repair*. All methods are evaluated on the same 67 tasks with 10 candidates each. Because rejection sampling and repair need diverse draws rather than beam output, the ordinary control uses the temperature-sampling decoding protocol (temperature 0.8, top- p 0.95), so its scores (14.93/23.88) differ from the beam-10 row of Tab. 2 (13.43/32.84); the TyFlow-2B row is identical in both tables.

Revised manuscript, Sec. 6.2.3 (p. 28) – reuse and adaptation of SynCode

For SynCode, we reuse its released grammar-masking code and Java grammar with T5Gemma2 on MBJP. At each step it masks the tokens that would violate a given grammar, so every output is syntactically well-formed.

Revised manuscript, Sec. 6.2.3 (p. 28) – reuse and adaptation of Repilot/JDT

For Repilot, we reuse its modified Eclipse JDT completion engine and adapt it to T5Gemma2 on MBJP. During generation, the partial Java program is maintained as a JDT document, and completion queries guide token pruning. Following the original policy, punctuation and keywords bypass these queries.

Revised manuscript, Sec. 6.2.3 (pp. 28–29) – the outcome and the Type Explicitness conclusion

None of the four external baselines significantly improves functional correctness over the ordinary control: rejection sampling adds three solved tasks at pass@10 (19 vs. 16 of 67) and one at pass@1 (11 vs. 10; McNemar $p = 1.00$ and $p = 0.25$); SynCode, Repilot, and iterative repair leave the solved sets exactly unchanged (10/67 and 16/67) [...] TyFlow-2B solves 17/67 at pass@1 and 29/67 at pass@10 with 3 of 670 candidates failing to compile, the best of all compared methods. The evaluated external mechanisms constrain or revise generated programs without updating the underlying model parameters, whereas TyFlow explicitly encodes the typing rules into the model and helps it better learn the program distribution, supporting the value of the Type Explicitness property in Sec. 1.

Comment 3.2.

Missing comparison against iterative refinement approaches: Compared to rejection sampling, iterative refinement approaches regenerate the program with feedback (in this case, the typing failure feedback), which is strictly stronger than rejection sampling as it incorporates typing failure feedback. To demonstrate TyFlow's robustness, the authors should compare it with this line of work.

Response.

Thanks for this comment. Iterative compiler repair is now one of the 5 compared methods: javac diagnostics are fed back to the model, which regenerates the candidate for at most 2 rounds; the repair agent is the un-fine-tuned T5Gemma2-2B, since the fine-tuned model never learned to consume diagnostics. We cap repair at 2 rounds per candidate to bound the additional generation cost and stop earlier if the candidate compiles.

Compiler feedback improves compilability in our experiments: the number of candidates that fail to compile falls from 175 to 116 out of 670. However, none of the 59 candidates repaired to compile passes the functional tests, so the solved sets remain unchanged at 10/67 for pass@1 and 16/67 for pass@10. Compiler diagnostics identify compilation errors, but do not establish whether a program implements the intended behavior. The feedback therefore helps repair compilation failures without necessarily correcting the program's functionality.

Rejection sampling explores additional candidates and can find functionally correct programs through repeated generation. In our experiments, it adds 1 solved task at pass@1 and 3 at pass@10, although neither gain is statistically significant. These results show that the additional compiler feedback is useful for compilability, but does not automatically lead to higher functional correctness than repeated sampling. And TyFlow performs better than these two methods since our representation helps model in better understanding the program structure and type reasoning process.

Changes made.

- **Added** Sec. 6.2.3 (pp. 28–29): the repair protocol (javac feedback, at most 2 rounds, un-fine-tuned agent), its measured outcome, and an explanation of why compiler feedback improves compilability without adding solved tasks.

Revised manuscript, Sec. 6.2.3 (p. 28) – the iterative-repair protocol

Iterative compiler repair feeds javac diagnostics back to the model and regenerates the candidate for at most two rounds; the repair agent is the un-fine-tuned T5Gemma2-2B, since the fine-tuned model never learned to consume diagnostics. We cap repair at two rounds per candidate to bound the additional generation cost and stop earlier if the candidate compiles.

Revised manuscript, Sec. 6.2.3 (pp. 28–29) – improved compilability without additional solved tasks

[. . .] Iterative repair reduces compilation failures from 175 to 116 of 670 candidates, but none of the 59 candidates repaired to compile passes the functional tests. Compiler diagnostics help resolve compilation errors, but do not establish whether a program implements the intended behavior, explaining why improved compilability does not translate into additional solved tasks here.

Comment 3.3.

The TyFlow system requires multiple LLM calls to obtain all synthesis decisions. To show the practicality of TyFlow, it is important to provide an analysis of the system’s cost for generating the program.

Response.

Thanks for this comment. TyFlow unifies the generation of all synthesis decisions into one LLM call with multiple output tokens, much like a vanilla encoder–decoder call. We will first clarify the base case of our LLM query process, then explain what our method adds on top of it and provide an analysis of the system’s cost.

(1) The base case is a single encoder–decoder call. The encoder processes the problem description once, and the decoder generates the output token by token, reusing the encoder’s representation throughout. There are no repeated LLM calls to separate models.

(2) What our method adds on top of this base. During decoding, we add syntactic pruning (checking each generated token against the grammar), type pruning (checking completed AST fields against the type system), and a dynamic typing context (re-encoding the synthesis goal as it evolves). Each of these adds moderate per-step cost, while the encoding of the static problem description is reused throughout generation.

(3) The measured cost. For the 220M model on SuFu, we measure the runtime of each component: starting from the base model without checks, we add syntactic pruning, type pruning, and the dynamic typing context in turn. Average time per decoding step (extending the active hypotheses by 1 token) is 31.1 ms for the base, 32.4 ms with syntax checks, 43.7 ms with type checks added, and 59.6 ms with all components. Average time per task follows the same configuration order: 5.70, 5.34, 11.07, and 15.62 s. Syntax checks add little per-step cost and reduce the number of decoding steps, so total task time decreases slightly; type checking and re-encoding the updated context account for the larger increases.

Changes made.

- **Added** Sec. 6.2.2 (pp. 27–28): per-step and per-task runtime for the 4 configurations (Table 4).
- **Added** Sec. 6.2.3 (p. 28): the computational budget of each compared external method.

Revised manuscript, Sec. 6.2.3 (p. 28) – the rejection-sampling budget

Rejection sampling repeatedly draws candidates and keeps those that pass Java compilation, for up to ten ten-draw rounds (100 candidates) per task.

Revised manuscript, Sec. 6.2.2, “Runtime cost” (p. 28) – the runtime measurements

Syntactic pruning adds little (roughly +4%); type pruning and the dynamic typing context cost considerably more, as the former invokes the type checker at every step and the latter additionally re-encodes the evolving synthesis goal.

Comment 3.4.

Limited Java Performance: Although TyFlow brings significant improvement on SuFu, its performance boost on Java is marginal (from 10.45% to 11.94%).

Response.

Thanks for this comment. We explain below why the improvement is smaller at pass@1 and how the other results help assess its value.

(1) Why the Java gains differ from SuFu. TyFlow improves code generation by preventing type errors during generation. Java is a mainstream programming language for which the pretrained baseline already has a strong foundation, whereas SuFu is a low-resource language. Consistent with this difference, the 220M Java baseline produces fewer compilation errors than the SuFu baseline (CER 35.52% versus 83.10%). This leaves fewer errors for our type constraints to address and therefore less room for improvement on Java.

(2) The value of the Java improvements. For a mainstream language such as Java, the improvement is substantial given that TyFlow and the baseline use the same training data. At 220M, TyFlow improves pass@10 by 7.46 percentage points (35.7% relative), a larger absolute gain than the 2.80-point improvement reported by GrammarT5-base over CodeT5-base on Java CONCODE, also at 220M.⁵ Gains of this size are typical for retraining-based approaches on Java: on CONCODE, AST-T5 improves the exact match of CodeT5-base by 0.6 points, and StructCoder by 0.1.⁶ Our expanded experiments confirm this on 2 additional Java benchmarks: at 2B, pass@1 improves from 31.25% to 50.00% on HumanEval-Java and from 19.42% to 30.10% on TransCoder-GFG, with all 4 metrics improving on both benchmarks (pass@10 from 37.50% to 56.25% and from 33.01% to 46.60%), and the per-language significance analysis finds all 4 Java differences statistically significant (App. D).

⁵Qihao Zhu et al. “GrammarT5: Grammar-Integrated Pretrained Encoder-Decoder Neural Model for Code.” ICSE 2024, Table 4. These gains are measured on different tasks and metrics.

⁶Linyuan Gong, Mostafa Elhoushi, and Alvin Cheung. “AST-T5: Structure-Aware Pretraining for Code Generation and Understanding.” ICML 2024: in its unified CONCODE comparison, the exact match is 22.3 for CodeT5-base, 22.9 for AST-T5, and 22.4 for StructCoder. Sindhu Tipirneni, Ming Zhu, and Chandan K. Reddy. “StructCoder: Structure-Aware Transformer for Code Generation.” ACM TKDD 2024. These gains are measured on different tasks and metrics.

Changes made.

- **Added** Sec. 6.2.1 (p. 25): the Java results at both model scales and the explanation of the 220M gains across metrics.

Revised manuscript, Sec. 6.2.1 (p. 25) – the Java results at both scales

TyFlow improves code generation accuracy at both scales and on every benchmark: pass@1 rises from 24.14% to 37.93% (SuFu) and from 10.45% to 11.94% (MBJP) at 220M, and from 25.86% to 36.21% (SuFu), 13.43% to 25.37% (MBJP), 31.25% to 50.00% (HumanEval-Java), and 19.42% to 30.10% (TransCoder-GFG) at 2B; pass@10 improves by 7.46–20.69 points on the same six benchmarks.

Revised manuscript, Sec. 6.2.1 (p. 25) – the Java gains across correctness and compilation metrics

At 220M, the Java baseline has a lower CER than the SuFu baseline (35.52% versus 83.10%), leaving fewer compilation errors for type constraints to address. On MBJP, TyFlow raises pass@10 from 20.90% to 28.36% and reduces CER to 1.53%; the smaller pass@1 gain indicates that finding a correct candidate does not always make it the top-ranked output. The failure analysis in Appendix C identifies ranking errors and compilable but incorrect programs among the remaining failures.

Comment 3.5.

Limited model choices: The TyFlow system was mainly evaluated on small models. Evaluation on larger models (decoder-only, billions of parameters) is unexplored. The authors should consider extending the evaluation to larger models.

Response.

Thanks for this comment. Our evaluation has included the experiments at the 2B scale, where the baseline T5Gemma2-2B and TyFlow-2B share the same training data and procedure (Sec. 6.2, Table 1, p. 26). At this scale, training on decision sequences rather than plain code improves every metric on every benchmark, reflecting the effectiveness of our decision-sequence representation. TyFlow improves consistently over its baselines even though the 2B baseline commits fewer type errors than the 220M baseline.

For scaling beyond 2B, the natural target is a decoder-only model, since the largest open models use this architecture. The representation carries over without essential change: a decoder-only model receives the same information (the task specification, the current typing context, and the decision tokens generated so far) and predicts the same next decision token, while the synthesis engine that applies decisions and checks constraints is architecture-independent. What needs adaptation is how the dynamic typing context reaches the model: in the current encoder-decoder design, the encoder processes the changing synthesis goal separately from the decision sequence, whereas a decoder-only model must append this context to the decision tokens generated so far. Since the context changes after every decision, this appended segment has to be updated at every step and its key-value entries recomputed, which raises an engineering cost in the decoding procedure (Sec. 6.3, p. 30).

Moreover, encoder-decoder and decoder-only models of comparable size have repeatedly been found to perform similarly: the T5 ablations show only modest differences between the two architectures at matched budgets,⁷ and instruction-finetuned encoder-decoder models remain competitive with same-size or larger decoder-only models.⁸ We therefore consider the 2B results representative of what the representation contributes at a given scale; training and evaluating a decoder-only instantiation remains future work.

Changes made.

- **Added** Sec. 6.2.1 and Table 3 (p. 26): three prompted decoder-only models on all 4 benchmarks, to contextualize absolute performance (also requested under AE-4 and R2-W3).
- **Added** Sec. 6.3 (p. 30): the requirements for a decoder-only adaptation.

Revised manuscript, Sec. 6.2 (p. 26) – improvements at both trained scales

TyFlow improves code generation accuracy at both scales and on every benchmark: pass@1 rises [...] from 25.86% to 36.21% (SuFu), 13.43% to 25.37% (MBJP), 31.25% to 50.00% (HumanEval-Java), and 19.42% to 30.10% (TransCoder-GFG) at 2B; pass@10 improves by 7.46–20.69 points on the same six benchmarks. [...] TyFlow also drops CER to 0.00% on SuFu and 0.45% / 1.30% / 2.75% on the three Java benchmarks.

Revised manuscript, Sec. 6.3 (p. 30) – the decoder-only adaptation path

The decision-sequence representation and synthesis rules can be used unchanged with a decoder-only model. [...] This requires recomputing the context’s key-value entries at each step and adapting cache management to the changing context. Implementing this decoding procedure and evaluating its runtime are directions for future work.

Comment 3.6.

Generalizability concern: TyFlow relies strongly on the typing system of the language, but how would it work on a dynamically typed language such as Python? Thus, the author should acknowledge this in the threat to validity to make this paper more robust.

Response.

Thanks for this comment. TyFlow can in principle be applied to statically checkable properties of a dynamically typed language such as Python. The key requirement is a suitable set of checking rules, rather than whether the language is classified as statically or dynamically typed. For Python, we could select a set of statically checkable typing rules and express them in our supported CHC form. Derivations recovered under these rules would provide training decision sequences, while the corresponding synthesis rules would ensure that generated programs satisfy the modeled checks.

⁷Colin Raffel, Noam Shazeer, Adam Roberts, et al. “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer”. JMLR 21, 2020.

⁸Hyung Won Chung, Le Hou, Shayne Longpre, et al. “Scaling Instruction-Finetuned Language Models”. 2022.

If these rules faithfully capture a static checker’s checks for the selected subset, the generated programs would pass those checks. This guarantee concerns the modeled static properties, not all runtime behavior. Our current experiments cover Java and SuFu; implementing the Python rules and derivation extraction and evaluating the adaptation remain future work. Sec. 6.3 now explains this extension path and the scope of the guarantee.

Changes made.

- **Added** Sec. 6.3, “Type-system expressiveness” (p. 30): the extension through statically checkable typing rules, automatic derivation extraction, the resulting guarantee, and the remaining implementation and evaluation work.

Revised manuscript, Sec. 1, “Data Usability” (p. 3) – the Data Usability property

The decision sequences of S and source code should be automatically and bidirectionally convertible, enabling both the extraction of decision sequences from existing programs and the reconstruction of programs from generated decision sequences.

Revised manuscript, Sec. 6.3, “Type-system expressiveness” (p. 30) – the extension path and scope of the guarantee

This extension path also applies to statically checkable properties of dynamically typed languages such as Python: we can select a statically checkable subset and model its typing rules. Automatically recovering derivations under these rules would provide training decision sequences, while the corresponding synthesis rules would ensure that generated programs satisfy the modeled static checks, rather than guarantee all runtime behavior. Implementing and evaluating these extensions remains future work.

Other Changes beyond the Reviewers' Requests

- **Updated experimental results.** The revised tables report updated experimental results, with the per-language significance analysis in Appendix D computed from these runs. We retrained the models and reran the evaluations because the original model checkpoints and evaluation records were lost during our migration to a new server. The main comparative findings remain unchanged: TyFlow improves over its corresponding baseline on all 4 metrics in each of the 4 original language–model–scale comparisons.
- **Errata.** Algorithm 1 now composes substitutions as $\sigma_{i-1} \dots \sigma_0 \sigma$; the statement after Lemma 4.1 reads $P(\theta(\bar{t}))$; and the proof of Lemma 4.4 defines $\theta_c = \sigma_n \dots \sigma_0 \sigma$. The method and the theorems are unchanged.
- **FSP definition.** Sec. 6.1.3 (p. 25): a task with no correct candidate contributes the candidate-list length, a convention the submitted text left implicit.
- **Prompt templates.** App. B now shows the concrete prompt template used by the evaluation: an example MBJP prompt (imports, class skeleton, and Javadoc) and an example SuFu prompt (the task description followed by the given program prefix of inductive definitions and library functions).

We hope that the revision addresses all concerns and look forward to the reviewers' feedback.

Sincerely,
The Authors